# 1 Introduction

## 1.1 Motivation

Recent advances in foundation models have substantially reshaped the role of neural systems in modern software. Large pretrained models can now be adapted to a wide range of downstream tasks, domains, and interaction settings with comparatively modest additional supervision, prompting, or tuning [2, 9, 11]. For language models (LMs), this shift is especially important. They are no longer used only as benchmarked text predictors, but increasingly as coding assistants, review tools, retrieval-augmented components, and engines for agentic workflows embedded in real development and decision-making pipelines [12, 22]. In these settings, the model is not a disposable experimental artifact. It is a persistent component of a broader software ecosystem whose behavior must remain useful, reliable, and maintainable over time. This thesis therefore treats language models not merely as objects of evaluation, but as *living engineering artifacts* whose capabilities, limitations, and maintenance costs matter in practice.

Once an LM is integrated into real workflows, its strengths and weaknesses become practical engineering concerns. Beneficial capabilities can be reused across tasks and interfaces, but so can outdated knowledge, brittle reasoning patterns, unsafe tendencies, and hard-to-maintain behaviors [2]. A model that hallucinates an obsolete API today may reproduce the same defect tomorrow in code synthesis, later in automated review, and again in an agentic repair loop. Likewise, a model that exhibits unstable reasoning or undesirable behavioral drift can propagate those problems across downstream services. The engineering problem is therefore not limited to training and deployment. It also includes understanding the model, maintaining it after deployment, and intervening when recurring failure modes begin to affect real use.

**Engineering Foundation Models as Maintainable Artifacts** Software engineering provides a natural starting point for this problem. Over several decades, it has developed systematic methods for specifying software, analyzing its structure, testing its behavior, localizing faults, and maintaining deployed systems [1]. Research on foundation models has likewise produced essential surrounding practices, including data and training pipelines, evaluation harnesses, deployment infrastructure, and MLOps lifecycle support [8]. These practices are indispens-

able, but they primarily organize work *around* the model. By contrast, this thesis focuses on the next step: treating the model itself as a maintainable artifact that can be analyzed, repaired, and made easier to maintain over time.

The need for explicit model-oriented maintenance becomes clearer when foundation models are compared with conventional software. A software component is grounded in explicit artifacts such as source code, interfaces, requirements, tests, and maintenance history. These do not eliminate complexity, but they provide a relatively stable basis for diagnosis, validation, and repair. A foundation model is different. Its functional content is learned rather than directly written: what the model knows, how it tends to reason, and which behavioral tendencies it exhibits are distributed across parameters, activations, and latent states [2, 14]. Benchmarks, task suites, and safety datasets provide important evidence about behavior, but they do not fully specify what the model has learned, how a particular output is produced, or how a local intervention will affect other capabilities. This limitation is the central *specification gap* between engineering conventional software and engineering learned models.

If language models are to be treated as important engineering artifacts, this specification gap must be addressed at the model level rather than merely worked around. Prompts, wrappers, filters, retrieval modules, and broad retraining remain important parts of the engineering toolbox, but they do not eliminate the need for direct maintenance of the model itself. When such maintenance is feasible, it offers at least three practical advantages. First, it improves *reusability*: a repair applied once to the model can benefit many downstream applications, interfaces, and workflows simultaneously, rather than being reimplemented separately in each application layer. Second, it improves *reliability*: recurring reasoning failures, unsafe tendencies, and persistent error patterns can be treated as maintainable system defects rather than tolerated as isolated bad outputs. Third, it improves *maintainability*: organizations must repeatedly accommodate changing APIs, local policies, domain conventions, proprietary knowledge, and evolving risk constraints, and targeted model-oriented interventions can often be updated and validated more systematically than repeated rounds of broad retraining. As foundation models become embedded in increasingly consequential workflows, their maintenance can no longer remain an informal afterthought.

**LM Analysis, LM Repair, and LM Meta-repair** This thesis develops model-oriented maintenance through three closely connected activities: *LM analysis*, *LM repair*, and *LM meta-repair*. *LM analysis* refers to the systematic examination of a lan-

guage model's internal representations, computational transitions, components, and mechanisms to explain semantically relevant behavior and determine where and why intervention may be required. Building on this diagnostic foundation, *LM repair* denotes the targeted modification of an already trained model to correct a specified failure or recurring failure pattern while minimizing unintended effects on unrelated knowledge and capabilities. *LM meta-repair* operates at a higher level by improving the semantic, structural, and methodological conditions under which future analysis and repair are conducted, including the consistency of internal representations, the clarity of intervention interfaces, and the modularity of model functions. Together, these activities address three fundamental maintenance questions: how model behavior can be understood, how identified deficiencies can be corrected, and how the model can be made more amenable to subsequent maintenance.

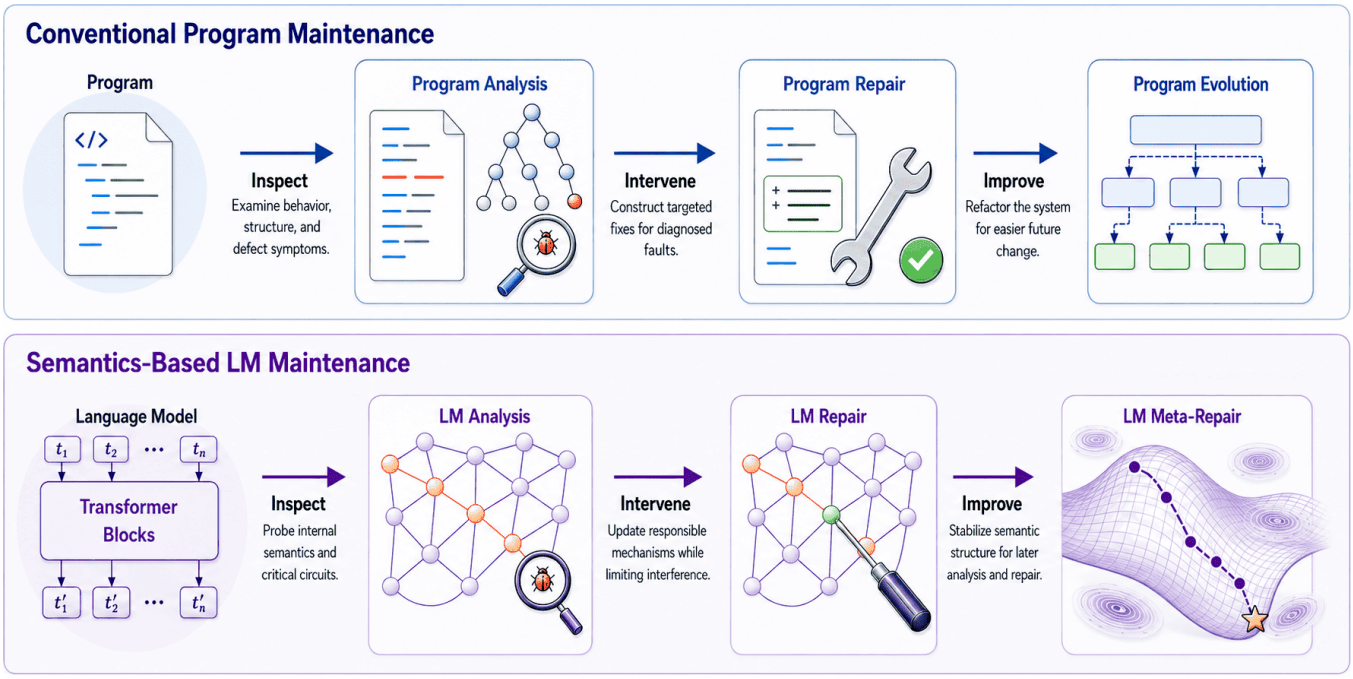

Figure 1.1: Analogy between conventional program maintenance and semantics-based language-model maintenance.

As shown in Figure 1.1, the thesis adapts the logic of conventional program maintenance to the learned and distributed setting of language models. In conventional software maintenance, engineers inspect a program to diagnose behavioral or structural defects, intervene by constructing targeted repairs, and improve the system through refactoring or evolution so that future changes are less costly. Semantics-based LM maintenance follows the same engineering logic, but the maintained artifact and the substrate of intervention differ: the object is a learned model rather than a written program; analysis targets internal semantic

structure, representations, and circuits rather than source-level control or data-flow structure; and repair modifies responsible mechanisms while constraining interference with unrelated capabilities. The final step, LM meta-repair, mirrors program evolution at a different level: it aims to stabilize and reorganize the semantic conditions under which future analysis and repair are performed.

An intuitive framing comes from the tripartite model of cognition, which distinguishes the *autonomous* mind, responsible for fast and default responses; the *algorithmic* mind, which provides the computational resources for deliberate reasoning and for maintaining decoupled or hypothetical representations; and the *reflective* mind, which sets goals, evaluates whether default responses should be questioned, and determines when analytic intervention should be initiated [16, 17]. The analogy is heuristic rather than literal. It does not imply that language models instantiate human cognition in any straightforward sense. Rather, it offers a useful conceptual scaffold for thinking about maintenance. At an abstract level, much LM behavior can be treated as a set of default computational tendencies distributed across parameters and representations. Analysis aims to make those tendencies legible. Repair aims to alter them when they become undesirable. Meta-repair aims to improve the conditions under which later diagnosis and intervention are performed. On this view, the tripartite framing organizes the thesis around three connected concerns: default behavior, targeted intervention, and improved capacity for future intervention.

**Semantics as the Operational Medium** To make this maintenance chain operational, the thesis treats *semantics* as the medium through which external requirements can be related to internal model computation. Engineers typically formulate failures in semantic terms: the model should know a fact, distinguish concepts, sustain a valid line of reasoning, avoid risky behavior, or produce code that satisfies particular constraints. The model, however, realizes these requirements through token interfaces, activations, residual states, attribution paths, gradients, and parameters. Because foundation models do not expose program-like specifications, semantics provides a practical medium for connecting external engineering requirements to internal model computation. In this sense, semantics is not merely a way of describing model behavior after the fact; it is the operational medium through which analysis, intervention, and validation can be aligned.

Accordingly, LM analysis in this thesis is not a generic post-hoc explanation, but a semantically grounded, mechanistic investigation that connects observed failures to internal computation. Recent advances in mechanistic interpretability

support this goal through neuron attribution, path tracing, circuit analysis, and residual-stream analysis [4, 14, 21]. Semantic grounding also makes repair more practical: it is not enough to locate a signal in the network; its semantic function must also be understood so that any intervention remains targeted, interpretable, and compatible with other capabilities. LM repair therefore refers to the targeted modification of an already trained model to correct a specific failure while preserving unrelated knowledge and functionality. It is distinct from broad retraining, generic fine-tuning, and factual knowledge editing [19, 20]. Although repair is difficult because model behavior is distributed across redundant representations, residual pathways, and large matrix transformations, this structure can also expose failure-related features that are suitable for targeted intervention [3]. LM analysis and repair must therefore be developed together. LM meta-repair extends this process by using repeated maintenance experience to improve the model's maintainability. Problems such as semantic entanglement, weak cross-layer comparability, update interference, and poor functional separation can make later repairs harder. Closely related to intrinsic interpretability [5], LM meta-repair feeds these maintenance requirements back into model architectures, training objectives, analytical methods, and internal organization, making future analysis and repair more modular, stable, and efficient.

**A Motivating Example** Modern software development is becoming more conversational and increasingly shaped by tools built on foundation models. These systems are used not only for code completion, but also for synthesis, refactoring, review, debugging, test generation, patch construction, and the coordination of complex development workflows [7, 10]. Recent research on *agentic coding* shows a broader move from AI-assisted programming to software workflows that are increasingly mediated by LMs, where AI systems take a more autonomous role in planning, execution, testing, and iterative refinement [6, 7, 15]. Code generation therefore offers a particularly visible instance of a broader maintenance problem. Its outputs are often directly executable, integrate easily into IDE and pipeline workflows, and tend to fail in recurring semantic ways, including API misuse, flawed control flow, inadequate error handling, insecure implementation patterns, and improper assumptions about executation environment. When a model repeatedly produces buggy, outdated, or risky code, repairing each generated program after the fact is not enough. Conventional program repair operates on individual artifacts: it may fix a specific output, but it does not change the model behavior that reproduces similar failures across tasks. When the source of the problem lies in the LM itself, for example in stale API knowledge, brittle

reasoning about program logic, or a persistent tendency to emit insecure or fabricated implementations, LM analysis and repair become a practically necessary way to improve code quality at its source.

This example clarifies why the thesis studies analysis, repair, and meta-repair together, and why it characterizes failures in terms of *knowledge*, *cognition*, and *behavior*. Analysis asks what aspects of the model should be examined: not only outputs and benchmark scores, but also semantic features, neuron activations, attribution paths, residual trajectories, layerwise roles, and cross-model semantic correspondences when these help explain computation. Repair asks what should be changed: outdated or distorted knowledge, brittle reasoning processes, and undesirable behavioral tendencies. In code generation, these correspond to incorrect or obsolete facts, such as deprecated APIs or incompatible dependency constraints; flawed intermediate reasoning about program semantics, specifications, or execution; and persistent behaviors such as overconfident fabrication, insecure default choices, or the recurrent production of vulnerable code. These categories are not intended as a rigid ontology, but as a practical engineering vocabulary for recurring failures. The same categories also arise beyond coding, for example when models must update stale knowledge, resist retrieval pollution, suppress dangerous behavioral drift, or remain reliable in increasingly agentic deployments [13, 18]. Meta-repair then asks what should be improved to make subsequent intervention easier: semantic interfaces, diagnostics, training-time constraints, and other aspects of model structure that determine whether maintenance is well defined in the first place. As LM analysis and repair develop further, future work will also move beyond observed *failures* to study the underlying *faults* in LMs, since even one defect can constantly causes failures.

## 1.2 Research Questions

This thesis examines *semantics-based LM analysis, repair, and meta-repair* within a unified maintenance framework. The research questions below are not intended to serve simply as an outline of individual chapters. Instead, each isolates a distinct aspect of the maintenance challenge: how semantics can be used to analyze LM mechanisms, how such analysis can inform targeted repair, and how language models can be improved to better support later repair.

***RQ1: How can semantics provide a practical basis to analyze LM mechanism?***

A principled approach to LM repair first requires an account of model computation in semantic terms: what internal states represent and how those states

evolve during processing. This question establishes the analytical foundation of the thesis. It asks whether semantic structure can be made sufficiently explicit to support engineering decisions. More specifically, it considers whether latent states can be compared as semantic objects, whether layerwise transitions can identify where meaningful computation occurs, and whether semantic reference can remain stable across layers or even across models.

***RQ2: How can semantically grounded mechanistic analysis support LM repair?***

Once the relevant semantic structure has been identified, the next challenge is to translate that analysis into concrete repair procedures. The focus here is on modifying model behavior under practical constraints, including limited data, limited computation, and minimal unintended side effects. This involves determining *where* to intervene, *how* to compute the update, and *how* to preserve unrelated capabilities while improving the target behavior. The central issue is not simply whether a model can be changed, but whether it can be repaired selectively, efficiently, and reliably enough to support practical maintenance.

***RQ3: How can language models be improved to support the needs of LM repair?***

Conventional repair addresses failures that are already evident in model behavior. Meta-repair operates at a different level. It asks how a model's semantic property and the analytical framework used to study it can be improved so that future repair becomes easier, safer, and more systematic. This includes developing stable token interfaces and explicit relations between the input embedding and output projection. In this respect, meta-repair intersects with intrinsic interpretability. More broadly, it asks how future language models can be made repair-friendly, rather than merely repairable through repeated intervention.

## 1.3 Thesis Overview

We summarize the overall structure of the thesis, as shown in Figure 1.2. The thesis is organized as a progression from semantics-based LM analysis to LM repair and, finally, LM meta-repair. In this progression, Chapters 3 and 4 address RQ1, Chapters 5 and 6 address RQ2, and Chapter 7 addresses RQ3. Chapter 2 provides the necessary background and Chapter 8 concludes the thesis.

Chapter 2 reviews the literature most relevant to the thesis, covering neural language models, mechanistic interpretability, model adaptation and editing, continual learning, neural network repair, and automated coding tasks. Its role is to situate the thesis problem and to clarify the reasons why semantics-based analy-

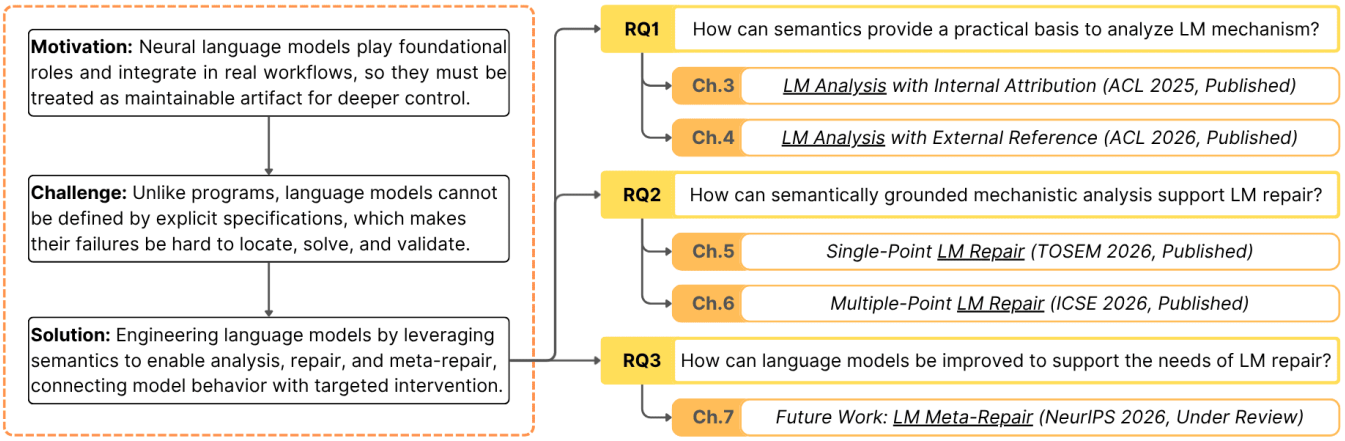

Figure 1.2: Overview of the thesis structure.

sis and maintenance should be studied together.

Part I, *Semantics-Based LM Analysis & Repair: Foundations*, addresses RQ1. Chapter 3 studies LM analysis with internal attribution by modeling layerwise hidden-state evolution as semantic transition and using attribution to localize semantically consequential computation for efficient adaptation. The work in this chapter was published at ACL 2025. Chapter 4 further extends the analysis to external semantic reference, examining latent semantic alignment across models and showing how semantic structure can support comparison and transfer without relying on brittle parameter-level correspondence. The work in this chapter was published at ACL 2026.

Part II, *Semantics-Based LM Analysis & Repair: Applications*, addresses RQ2 and RQ3. Chapter 5 introduces single-point LM repair through a semantics-based neuron-level framework for correcting one recurring failure pattern at a time under limited data and computational budgets. The work in this chapter was published in TOSEM 2026. Chapter 6 generalizes this setting to multiple-point LM repair through a semantics-guided optimization framework that coordinates multiple interacting updates while controlling sparsity, side effects, and forgetting. The work in this chapter was published at ICSE 2026. Chapter 7 then turns to LM meta-repair through Pseudo-Inverse Tying, which gives the input embedding and output projection a consistent token interface for continual pretraining and later model updates. The work in this chapter is currently under review.

Chapter 8 concludes the thesis by synthesizing the main findings and contributions of the semantics-centered methodology developed in the preceding chapters and by discussing its implications for the analysis, repair, and long-term maintenance of language models.

## 1.4 Contributions

Relative to the motivating goal of treating foundation models as maintainable engineering artifacts, the main contributions of this thesis are as follows:

- **A formulation of maintenance for language models.** The thesis frames LMs as living artifacts that require analysis, repair, and meta-repair, rather than merely improved surrounding workflows. This framing clarifies why maintaining deployed LMs matters not only for downstream services, but also for self-maintained ecosystems and LM-based agent systems.
- **A semantics-centered view of maintenance.** The thesis develops a coherent account in which semantics serves as the operational medium relating requirements stated by users, observed model behavior, and internal computation. This perspective provides the conceptual basis for making mechanistic analysis and operations directly relevant to maintenance.
- **Methods for semantics-based LM analysis.** The thesis develops concrete analysis methods for identifying semantically meaningful supervisonal signals, either internal attribution for transition analysis or external reference for latent alignment. These methods show how semantic analysis can reveal where critical computation occurs and how semantics can be utilized.
- **Targeted repair methods for efficient LM maintenance.** The thesis formulates LM repair as a bounded maintenance problem, distinct from full retraining yet broader than classical factual editing. It then contributes two complementary repair methods for single-point and multiple-point failures. Together, these methods show how models can be corrected and customized.
- **A meta-repair perspective for improving later maintenance.** Beyond correcting individual failures, the thesis studies how later intervention can be made easier. By introducing Pseudo-Inverse Tying, it provides a consistent token interface between the input embedding and output projection, creating a more stable basis for subsequent analysis, maintenance, and repair.

## References


[1] Abran, A., Bourque, P., Dupuis, R., and Moore, J. W. *Guide to the software engineering body of knowledge-SWEBOK*. IEEE Press, 2001.

[2] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).

[3] El Husseini, A., Izza, Y., Genest, B., and Roychoudhury, A. Repairing llm executions for secure automatic programming. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering* (2026).

[4] Ferrando, J., and Voita, E. Information flow routes: Automatically interpreting language models at scale. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (2024), pp. 17432–17445.

[5] Gao, Y., Meng, Q., Zhou, Y., and Pan, L. Towards intrinsic interpretability of large language models: A survey of design principles and architectures. *arXiv preprint arXiv:2604.16042* (2026).

[6] Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., Wang, L., Yao, J., Liu, T., Cai, Y., et al. A survey of vibe coding with large language models. *arXiv preprint arXiv:2510.12399* (2025).

[7] Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., and Qiu, D. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216* (2025).

[8] Hewage, N., and Meedeniya, D. Machine learning operations: A survey on mlops tool support. *arXiv preprint arXiv:2202.10169* (2022).

[9] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556 10* (2022).

[10] Jiang, J., Wang, F., Shen, J., Kim, S., and Kim, S. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology 35*, 2 (2026), 1–72.

[11] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[12] Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., and Lou, Y. Large language model-based agents for software engineering: A survey. *ACM Transactions on Software Engineering and Methodology* (2024).

[13] Lynch, A., Wright, B., Larson, C., Ritchie, S. J., Mindermann, S., Hubinger, E., Perez, E., and Troy, K. Agentic misalignment: How llms could be insider threats. *arXiv preprint arXiv:2510.05179* (2025).

[14] Rai, D., Zhou, Y., Feng, S., Saparov, A., and Yao, Z. A practical review of mechanistic interpretability for transformer-based language models. *arXiv preprint arXiv:2407.02646* (2024).

[15] Sapkota, R., Roumeliotis, K. I., and Karkee, M. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443* (2025).

[16] Stanovich, K. *Rationality and the reflective mind*. Oxford University Press, 2011.

[17] Stanovich, K. E. Distinguishing the reflective, algorithmic, and autonomous minds: Is it time for a tri-process theory. *In two minds: Dual processes and beyond* (2009), 55–88.

[18] Turner, E., Soligo, A., Taylor, M., Rajamanoharan, S., and Nanda, N. Model organisms for emergent misalignment. *arXiv preprint arXiv:2506.11613* (2025).

[19] Wang, S., Zhu, Y., Liu, H., Zheng, Z., Chen, C., and Li, J. Knowledge editing for large language models: A survey. *ACM Computing Surveys 57*, 3 (2024), 1–37.

[20] Yao, Y., Wang, P., Tian, B., Cheng, S., Li, Z., Deng, S., Chen, H., and Zhang, N. Editing large language models: Problems, methods, and opportunities. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (2023), pp. 10222–10240.

[21] Yu, Z., and Ananiadou, S. Neuron-level knowledge attribution in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (2024), pp. 3267–3280.

[22] Zhang, Q., Fang, C., Xie, Y., Zhang, Y., Yang, Y., Sun, W., Yu, S., and Chen, Z. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223* (2023).